



```
In [4]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [5]: df = pd.read_csv(r"C:\Users\A.K SINGH\AppData\Local\Packages\5319275A.WhatsApp
```

```
In [6]: df.shape
```

```
Out[6]: (1460, 81)
```

```
In [7]: df.head()
```

```
Out[7]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	L
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	

5 rows × 81 columns

```
In [8]: pd.set_option('display.max_columns',None)
```

```
In [9]: df.head()
```

```
Out[9]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	L
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	

```
In [10]: pd.set_option('display.max_rows',None)
```

```
In [11]: df.head()
```

```
Out[11]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	L
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	

```
In [12]: num_cols = df.select_dtypes(include=['int64','float64']).columns
cat_cols = df.select_dtypes(include=['object']).columns
```

```
In [13]: print("\nNumerical Columns:")
print(list(num_cols))

print("\nCategorical Columns:")
print(list(cat_cols))
```

Numerical Columns:

```
['Id', 'MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageYrBlt', 'GarageCars', 'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal', 'MoSold', 'YrSold', 'SalePrice']
```

Categorical Columns:

```
['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2', 'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQual', 'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual', 'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature', 'SaleType', 'SaleCondition']
```

```
In [14]: print(df.isnull().sum().sort_values(ascending=False))
```

PoolQC	1453
MiscFeature	1406
Alley	1369
Fence	1179
MasVnrType	872
FireplaceQu	690
LotFrontage	259
GarageYrBlt	81
GarageCond	81
GarageType	81
GarageFinish	81
GarageQual	81
BsmtFinType2	38
BsmtExposure	38
BsmtQual	37
BsmtCond	37
BsmtFinType1	37
MasVnrArea	8
Electrical	1
Id	0
Functional	0
Fireplaces	0
KitchenQual	0
KitchenAbvGr	0
BedroomAbvGr	0
HalfBath	0
FullBath	0
BsmtHalfBath	0
TotRmsAbvGrd	0
GarageCars	0
GrLivArea	0
GarageArea	0
PavedDrive	0
WoodDeckSF	0
OpenPorchSF	0
EnclosedPorch	0
3SsnPorch	0
ScreenPorch	0
PoolArea	0
MiscVal	0
MoSold	0
YrSold	0
SaleType	0
SaleCondition	0
BsmtFullBath	0
HeatingQC	0
LowQualFinSF	0
LandSlope	0
OverallQual	0
HouseStyle	0
BldgType	0
Condition2	0
Condition1	0
Neighborhood	0

LotConfig	0
YearBuilt	0
Utilities	0
LandContour	0
LotShape	0
Street	0
LotArea	0
MSZoning	0
OverallCond	0
YearRemodAdd	0
2ndFlrSF	0
BsmtFinSF2	0
1stFlrSF	0
CentralAir	0
MSSubClass	0
Heating	0
TotalBsmtSF	0
BsmtUnfSF	0
BsmtFinSF1	0
RoofStyle	0
Foundation	0
ExterCond	0
ExterQual	0
Exterior2nd	0
Exterior1st	0
RoofMatl	0
SalePrice	0

dtype: int64

```
In [15]: df["LotFrontage"] = df["LotFrontage"].fillna(df["LotFrontage"].median())
df["MasVnrArea"] = df["MasVnrArea"].fillna(df["MasVnrArea"].median())
df["GarageYrBlt"] = df["GarageYrBlt"].fillna(df["GarageYrBlt"].median())
```

```
In [16]: df["Electrical"] = df["Electrical"].fillna(df["Electrical"].mode()[0])
df["MasVnrArea"] = df["MasVnrArea"].fillna(df["MasVnrArea"].mode()[0])
df["GarageType"] = df["GarageType"].fillna(df["GarageType"].mode()[0])
```

```
In [17]: null_percent = (df.isnull().sum()/len(df))*100
drop_cols = null_percent[null_percent > 40].index
```

```
In [18]: print("Dropped Columns:")
print(drop_cols)
```

Dropped Columns:
Index(['Alley', 'MasVnrType', 'FireplaceQu', 'PoolQC', 'Fence', 'MiscFeature'],
dtype='object')

```
In [19]: df.drop(columns=drop_cols, inplace=True)
```

```
In [20]: num_cols = df.select_dtypes(include=["int64","float64"]).columns
cat_cols = df.select_dtypes(include=["object"]).columns
```

```
In [21]: print(df.duplicated().sum())
```

0

```
In [22]: df = pd.get_dummies(df,
                             columns=cat_cols,
                             drop_first=True)
```

```
In [23]: print(df.shape)
```

(1460, 231)

```
In [24]: X = df.drop("SalePrice", axis=1)
y = df["SalePrice"]
```

```
In [25]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

```
In [26]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.20,
    random_state=42
)
```

```
In [27]: print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)

print("Data Cleaning & Preprocessing Completed Successfully")
```

X_train: (1168, 230)

X_test : (292, 230)

y_train: (1168,)

y_test : (292,)

Data Cleaning & Preprocessing Completed Successfully

```
In [28]: df.head()
```

```
Out[28]:
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt
0	1	60	65.0	8450	7	5	2003
1	2	20	80.0	9600	6	8	1976
2	3	60	68.0	11250	7	5	2001
3	4	70	60.0	9550	7	5	1915
4	5	60	84.0	14260	8	5	2000

```
In [29]: X = df.drop('SalePrice', axis=1)
y = df['SalePrice']
```

```
In [30]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
In [31]: from sklearn.linear_model import LinearRegression
```

```
In [32]: lr = LinearRegression()
```

```
In [33]: lr.fit(X_train, y_train)
```

```
Out[33]:
```

▼ LinearRegression ⓘ ?

Parameters

	fit_intercept	True
	copy_X	True
	tol	1e-06
	n_jobs	None
	positive	False

```
In [34]: y_pred = lr.predict(X_test)
```

```
In [35]: print("R2 Score :", lr.score(X_test, y_test))
```

R2 Score : 0.6425586572167541

```
In [36]: from sklearn.metrics import mean_squared_error
```

```
mse = mean_squared_error(y_test, y_pred)

print("MSE :", mse)
```

MSE : 2741691699.082487

```
In [37]: rmse = np.sqrt(mse)

print("RMSE :", rmse)
```

RMSE : 52361.165944643435

```
In [38]: import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")

plt.title("Actual vs Predicted House Price")

plt.show()
```

